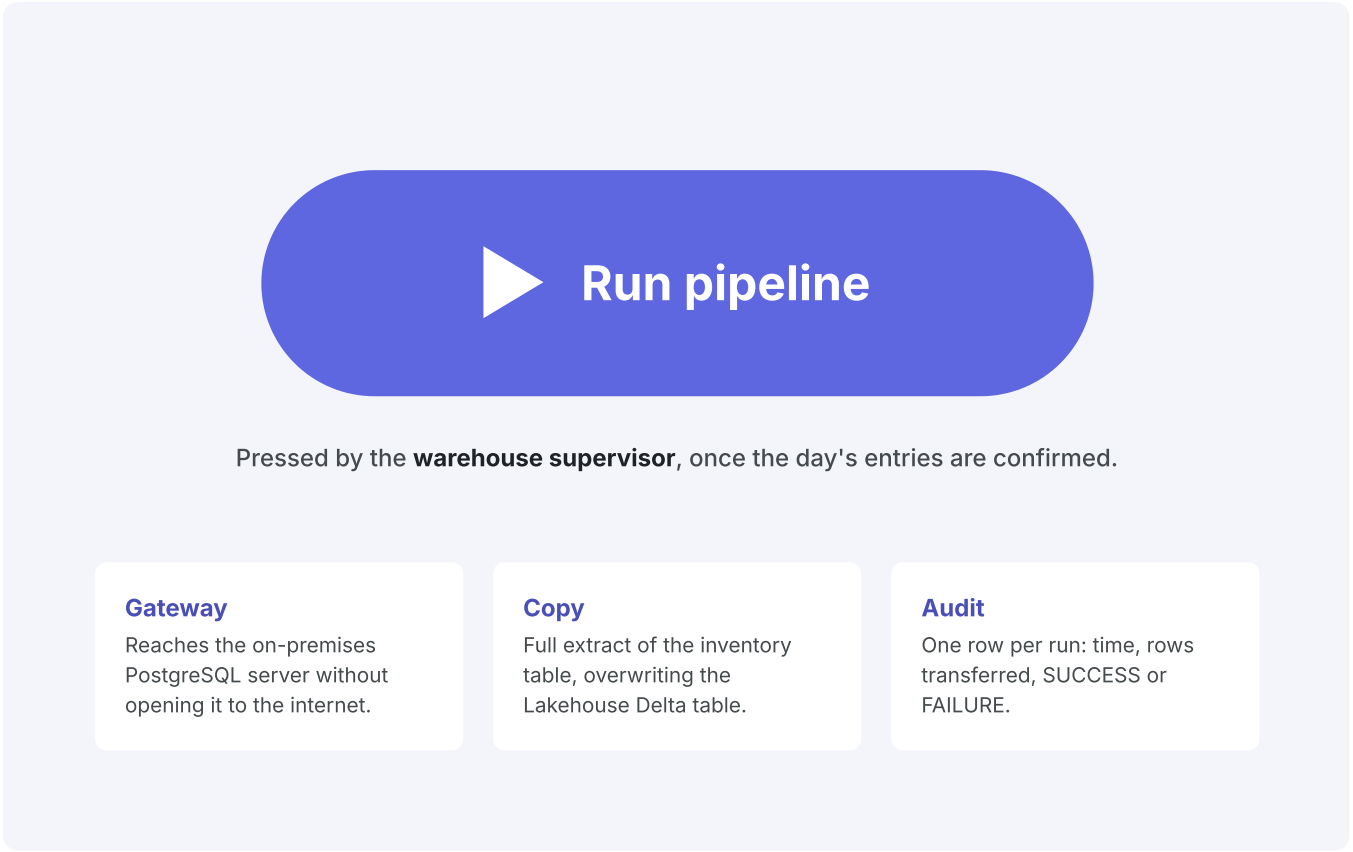


Inventory Data Migration.

A Microsoft Fabric case study in on-demand PostgreSQL ingestion through an on-premises data gateway, Copy activity orchestration and a governed Lakehouse Delta table, for a multi-warehouse retailer replacing manual CSV exports.

01 The Brief
02 The Diagnosis
03 The Strategy
04 The Mandate
05 Acceptance
06 Discussion and Lab

RetailCore Ltd
Multi-warehouse retail, PostgreSQL ingestion

Microsoft Fabric

SECTION 01 · THE BRIEF

Stop carrying data by hand.
Let it arrive on a trigger.

RetailCore Ltd runs twelve regional warehouses with about eighty thousand active SKUs. Warehouse staff keep the inventory by hand in a PostgreSQL database on an on-premises server, and the merchandising team uses it every day to decide replenishment and allocation. Getting that data into Microsoft Fabric is the single point of failure, and today that transfer is a person.

COMPLICATION

An analyst exports the table as a CSV each day and drops it in a shared network folder. Delays are unpredictable, one absence stops the feed, and nothing records when the data was last refreshed or how many rows moved.

RESOLUTION

A Fabric pipeline with a Copy activity and an on-premises data gateway moves the PostgreSQL table straight into the Lakehouse as a governed Delta table. The warehouse supervisor triggers it once the day's entries are confirmed, and every run leaves an audit record.

> *I cannot authorise a purchase order for 40,000 units if I do not know whether yesterday's stock count has been loaded. The manual upload has failed us three times this quarter alone.*
>
> HEAD OF MERCHANDISING · RETAILCORE LTD

EXHIBIT 01 · KEY ORGANISATIONAL METRICS

METRIC	VALUE	CONTEXT
WAREHOUSES	12 regional	Distributed locations feeding one central distribution centre.
ACTIVE SKUS	About 80,000	Unique stock-keeping units live at any time, from over 200 suppliers.
DAILY TRANSACTIONS	15,000 to 22,000	Manual entries recorded across the working day.
REPORTING CONSUMERS	3 functions	Merchandising, procurement and operations.

WHAT THE PIPELINE MUST DELIVER

01	**Reliability**	No manual CSV export. The gateway and pipeline do the transfer.
02	**Speed**	Transfer time drops from hours of manual effort to minutes of pipeline run.
03	**Compliance**	Every run leaves a timestamped audit record of rows transferred and run time.

SECTION 02 · THE DIAGNOSIS

A human as the integration layer.

This is not a data quality problem. The PostgreSQL table is kept carefully and is accurate. The failure is in the transfer between the database and the analytics layer: with no engineered connection, a person does the job, unreliably, without scale and with no audit trail.

EXHIBIT 02 · THREE FAILURE VECTORS

FAILURE	WHAT HAPPENS	MEASURE
AVAILABILITY	**If the analyst is absent or busy, no export happens, and merchandising works on stale data without knowing it.**	12 to 36 hours of reporting latency on a failed day
INTEGRITY	**Manual CSVs get truncated above size limits, garble special characters in product names, and silently drop columns when the source changes.**	3 failed exports this quarter
AUDITABILITY	**No record of when the last export ran, how many rows moved, or whether the report received what left PostgreSQL.**	0 audit records

THE ORGANISATIONAL TENSION

Procurement escalates freshness complaints to Data Engineering, which cannot see or fix the manual process. The supervisor cannot confirm the upload arrived. Engineers spend their time investigating instead of building.

WHY THE TRIGGER GAP MATTERS

Nothing tells the analytics layer the warehouse data is ready, so merchandising cannot tell if it sees today or yesterday. Tying the readiness confirmation to the transfer is the real design problem.

SECTION 03 · THE STRATEGY

From a pull model to a push model.

A Fabric pipeline replaces the person in the middle. The trigger moves from an analyst's memory to a deliberate business action: the warehouse supervisor, who knows when the day's entries are complete, starts the transfer, so readiness and transfer happen in one auditable step.

EXHIBIT 03 · OPERATING MODEL SHIFT

DIMENSION	CURRENT	TARGET
TRANSFER METHOD	**Manual CSV export by an analyst.**	Pipeline Copy activity.
TRIGGER	**The analyst's memory and availability.**	The warehouse supervisor runs the pipeline.
AUDITABILITY	**No machine record of transfers.**	One audit table row per pipeline run.
DATA FORMAT	**CSV with no schema.**	Delta table with ACID guarantees in the Lakehouse.
CONNECTIVITY	**File drop to a shared network folder.**	On-premises data gateway.
FAILURE VISIBILITY	**Found when reports look stale.**	Pipeline failure shown immediately in Fabric.

TWO ACTIVITIES

The Copy activity moves the data from PostgreSQL to the Lakehouse. A second activity writes the audit record of that move. Together they repeat what the analyst did, with no file handling and a machine-made record every run.

THE GATEWAY AS ENABLER

The on-premises data gateway reaches the PostgreSQL server without exposing it publicly. It runs as a service on the same server and opens outbound connections to Fabric. The one dependency: the server and gateway must be running when the supervisor triggers the pipeline.

SECTION 04 · THE MANDATE

A production transfer mechanism. Not a proof of concept.

You are the Lead Data Engineer, reporting to the Enterprise Platform Owner. Design, build and validate the pipeline that moves the PostgreSQL inventory table into the Lakehouse as a governed Delta table. You are accountable for the transfer, the audit record and the connection.

FULL EXTRACT, FULL OVERWRITE

Staff update records all day, and the same record may be corrected several times before the supervisor confirms. An incremental load on insert timestamps is unreliable for this source. A full extract followed by a full overwrite of the Delta table means the Lakehouse always matches PostgreSQL at the moment of the trigger.

EXHIBIT 04 · SOURCE SYSTEM ANALYSIS

ATTRIBUTE	DETAIL	NOTE
SOURCE SYSTEM	PostgreSQL	On-premises, on the central distribution centre server.
SOURCE TABLE	`inventory."InventorySnapshot"`	Schema inventory. The table name is quoted mixed case.
CONNECTION	On-premises data gateway	Installed on the source server.
EXTRACTION	Full extract	The whole table on every run, then overwrite.
ROW COUNT	About 670,000	80,000 SKUs, each held in about 70 percent of the 12 warehouses.
UPDATES	Continuous	Manual updates through business hours.
READINESS SIGNAL	Manual trigger	The warehouse supervisor runs the pipeline in Fabric.

EXHIBIT 05 · TARGET SCHEMA · SIX COLUMNS, ALL FROM SOURCE

COLUMN	DELTA TYPE	SOURCE FIELD AND RULE	ROLE
`SKU_ID`	string	sku_id. Not null in source. Key together with WarehouseCode.	Key
`ProductName`	string	product_name. Not null in source.	Mapped
`WarehouseCode`	string	warehouse_code, WH01 to WH12. Not null in source. Key together with SKU_ID.	Key
`StockOnHand`	int	stock_on_hand. Defaults to 0 in source.	Mapped
`LastUpdated`	timestamp	last_updated, a timestamp with time zone. Not null in source.	Mapped
`UnitCost`	decimal(10,2)	unit_cost. Two decimal places, no rounding.	Mapped

Delta does not enforce keys or NOT NULL. Check by query that each SKU_ID and WarehouseCode pair appears once and that the not-null columns hold no nulls. The audit record lives in its own table, not in these rows.

Scope. One source table, one pipeline and one audit table. The supervisor must be able to run the pipeline from the Fabric portal without engineering help, and the gateway connection must be reusable. Power BI refresh and extra tables are out of scope for this phase.

SECTION 05 · ACCEPTANCE

Four criteria. Confirmed by a replay test.

Each criterion is met in full or not met. Platform Engineering runs the test with the warehouse supervisor present, triggering the pipeline from the Fabric portal.

01	**Completeness**	The Lakehouse table is a row-for-row copy of the PostgreSQL table at run time. Its row count matches a direct count on the source in the same window.
02	**Integrity**	Every column has the Delta type in Exhibit 05. UnitCost keeps two decimal places without rounding, and LastUpdated keeps its time zone. A query finds no nulls in the not-null columns and no repeated SKU_ID and WarehouseCode pair.
03	**Governance**	Every run writes one row to the audit table: run time, rows transferred and status, SUCCESS or FAILURE. A separate activity writes it, so a failed copy shows up in the log.
04	**Idempotency**	Running the pipeline several times in a day adds no duplicate rows, because each run overwrites the whole table. The audit table records every run.

Validation. Trigger the pipeline and let it finish. Count rows in PostgreSQL and in the Delta table and confirm they match. Check the audit table has one new row with the right time and count. Trigger again without source changes and confirm the row count is unchanged.

SECTION 06 · DISCUSSION AND LAB

Seven questions. One pipeline.

Defend each answer with evidence from your design, not a generic platform description.

Q1 · Architecture ownership	Why is a pipeline Copy activity the right mechanism rather than a Python script on the local server pushing to the Lakehouse API? Which can the warehouse IT team maintain without data engineering help?
Q2 · Gateway reliability	What does the pipeline do when the gateway is unreachable? How does the supervisor learn it failed before merchandising finds stale data?
Q3 · Full extract or incremental	At about 670,000 rows a full extract and overwrite is fine. At what size would you switch to a watermark-based incremental load, and which column could be the watermark when staff update existing records?
Q4 · Platform trade-offs	Dataflow Gen2 or pipeline Copy activity for a relational source: which suits a non-technical supervisor who must trigger the transfer and diagnose failures alone?
Q5 · Governance maturity	What else should the audit table capture for a six-month lineage report, and how would you extend it without breaking the existing activity?

Q6 · Business value	If the pipeline removes all manual transfer failures, how much overstock write-off risk does it remove across twelve warehouses in the first quarter?
Q7 · Scalability path	Three more operational databases sit on the same server. How do you onboard them without duplicating the gateway setup?

LAB · BUILD IT PHASE BY PHASE

You are the Data Engineer on the Enterprise Platform Team. No file handling and no manual export at any stage.

Phase 01
Connect

Install the on-premises data gateway on the PostgreSQL server. In Fabric, create a connection that uses it, and check it by browsing `inventory."InventorySnapshot"` from the pipeline designer.

Output: gateway connection

Phase 02
Audit table

Create a Lakehouse Delta table `pipeline_audit_log` with run_timestamp, source_table, rows_transferred and execution_status. One row per run.

Output: audit table

Phase 03
Copy

Add a Copy activity: source the PostgreSQL table through the gateway connection, sink the Lakehouse Delta table with table action Overwrite. Check the mappings against Exhibit 05.

Output: Copy activity

Phase 04
Audit row

After the copy, write one audit row with the run ID, run time and rows copied from the Copy activity output. Use a Notebook activity, or a second Copy activity whose source query returns those values. A failed run writes a FAILURE row.

Output: audit row

Phase 05
Verify

Trigger the pipeline. Compare the Lakehouse row count with a direct count on PostgreSQL, and confirm the audit row. Trigger again and confirm the Delta row count is unchanged.

Output: matching counts

RISKS AND MITIGATIONS

RISK	WHAT HAPPENS	MITIGATION
GATEWAY UNAVAILABLE	**The server is off or the gateway service stops, and the pipeline fails at the connection step.**	Run the gateway as an automatic-start service, and alert the platform team by email on any failed run.
SOURCE SCHEMA DRIFT	**Warehouse IT adds or removes columns without notice, and the Copy activity errors or silently drops fields.**	Compare the source column count with a stored baseline before the copy.
SUPERVISOR DELAY	**The on-demand model depends on the supervisor starting the run.**	Name a second authorised operator per location and document the procedure.

MICROSOFT FABRIC BOOTCAMP BY COE

From a daily CSV by hand to a governed Delta on a trigger.

Give RetailCore an inventory feed it can trust: a gateway to the on-premises PostgreSQL server, a full extract and overwrite into a Lakehouse Delta table, an audit row on every run, and a pipeline the supervisor can trigger the moment the day's entries are done.

WHAT YOU DELIVER

A tested gateway connection, a Copy pipeline that overwrites the Lakehouse Delta table with the full inventory snapshot, and a pipeline_audit_log table with one row per run.

HOW YOU ARE EVALUATED

Four acceptance criteria: matching row counts, the Exhibit 05 types, an audit row on every run, and an unchanged count after a second trigger.

WHAT THIS PROVES

You can replace a human integration layer with a governed pipeline, defend a full extract over an incremental one, and reach an on-premises source without exposing it.

WHAT COMES NEXT

Onboard the three other databases through the same gateway. Switch to incremental loads when a table outgrows a full extract. Configure Power BI refresh on the Delta table.

INSTRUCTOR CONTACT

Surya Kumar · suryakumar01985@gmail.com